

Detailed Guide for Upcoming Quizzes

MongoDB (25 Most Common Operations)

You are expected to recognize, understand, and know when to use the following core MongoDB commands:

Insert Operations

- `insertOne()`
- `insertMany()`

Read Operations

- `find()`
- `findOne()`
- `find().limit()`
- `find().skip()`
- `find().sort()`
- `distinct()`
- `countDocuments()`

Update Operations

- `updateOne()`
- `updateMany()`
- `replaceOne()`
- `findOneAndUpdate()`
- `findOneAndReplace()`

Delete Operations

- `deleteOne()`
- `deleteMany()`

- `findOneAndDelete()`

Aggregation

- `aggregate()`

Index Operations

- `createIndex()`
- `dropIndex()`
- `getIndexes()`

Bulk & Collection Management

- `bulkWrite()`
- `renameCollection()`
- `drop()`
- `listCollections()`

Note: You do NOT need to memorize exact syntax, but you must understand what each operation does, when to use it and basic structure of the commands.

Mongoose Models & Basic Relationships

Study the following:

Mongoose Basics

- Connecting to MongoDB with `mongoose.connect()`
- What a **Schema** is
- What a **Model** is

Creating Schemas

Example:

```
const userSchema = new mongoose.Schema({  
  name: String,
```

```
age: Number,  
});
```

Defining Models

```
const User = mongoose.model("User", userSchema);
```

Performing CRUD with Models

- `Model.create()`
- `Model.find()`
- `Model.findOne()`
- `Model.updateOne()`
- `Model.deleteOne()`

Making Keys in Mongoose

Mongoose also gives every document an `_id` automatically.

You can create custom IDs:

```
const userSchema = new mongoose.Schema({  
  _id: String,  
  name: String  
});
```

You can create **unique keys** by adding:

```
email: { type: String, unique: true }
```

Basic Relationships (Population)

You must know:

- How to reference another model using `ObjectId`
- How to use `ref`
- How `populate()` works

Example:

```
postSchema = ({
  title: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: "User" }
});
```

React Basics:

1. useState Hook

Be able to:

- Create and update state
- Use previous state (`prev ⇒ prev + 1`)
- Understand how state updates affect re-rendering
- Implement toggles (true/false switching)

2. useEffect Hook

You must know:

- When useEffect runs
- How dependency arrays work
- How to make useEffect run when a specific value changes

3. useMemo Hook

Understand:

- What useMemo is used for
- How it improves performance
- Difference between useMemo and useEffect
- Difference between useMemo and useState

4. React.memo

Be confident about:

- What React.memo does
 - When to use it
 - How it differs from useMemo
-

5. Conditional Rendering

Practice:

- Rendering different UI based on state
 - Using ternary operators
 - Using && conditions
-

6. Event Handling

You should know how to:

- Use `onClick`, `onSubmit`, `onChange`
 - Call functions on events
 - Prevent default behaviour in forms
-

7. Controlled Components (Forms)

Practice:

- Two-way binding using useState
 - Handling input changes
 - Handling form submission
-

8. Props

You must know:

- Passing props to components
- Receiving props

- Displaying props inside components
-

9. Dynamic Rendering Using map()

Be able to:

- Render lists using `.map()`
 - Pass unique keys (`key={id}`)
 - Generate multiple components from arrays
-

10. Component Structure & Basic JSX

Make sure you're comfortable with:

- Functional components
 - Returning JSX with one parent element
 - Using expressions inside JSX
-